



Test

Controllare la correttezza del codice



Test

- Un test è un blocco di codice creato intenzionalmente per verificare se una certa porzione di codice funziona o meno
- Verificare funzionalmente i singoli componenti di un sistema è un modo efficace e pratico di creare e mantenere codice di alta qualità
 - Un test non può dimostrare l'assenza di errori, ma aiuta a generare confidenza nel sistema nel momento in cui questo viene messo in campo e aiuta a conservare la correttezza della base di codice, quando il progetto deve essere mantenuto nel tempo
 - E' molto complesso ri-organizzare su larga scale del codice in assenza di test di unità
- I benefici legati ad un uso intelligente e bilanciato dei test di unità nel software sono profondi
 - Nelle fasi di implementazione, test di unità ben scritti diventano una specifica informale dei componenti di un sistema
 - Nelle fasi di manutenzione, i test esistenti servono da freno alle regressioni nel codice, stimolando una correzione immediata

Test di unità

- Un **test di unità** valuta il comportamento di un **singolo componente** software (funzione, struttura dati, ...), indipendentemente dal resto del sistema
 - Sollecitandone sia il comportamento “tipico”, ovvero conforme all’uso per cui è stato progettato, sia quello agli estremi o oltre il suo previsto campo di utilizzo
- Tale sollecitazione viene esercitata tramite frammenti di codice che invocano il componente e verificano che esso risponda in modo atteso
 - Confrontando, ad esempio, i valori ritornati dalla funzione / contenuti nella struttura dati dopo la sollecitazione con valori/condizioni di errore attesi sulla base delle specifiche
- Il test cosgtituisce unn punto di ingresso che può sempre essere utilizzato per validare il componente
- I test di unità sono normalmente scritti ed eseguiti dallo sviluppatore
 - Presuppongono la conoscenza dei meccanismi interni al componente, allo scopo di verificare come questi reagiscono in presenza di casi limite
 - Il loro numero dipende dalla complessità ciclomatica del modulo testato
 - La loro qualità è alla base del processo di refactoring cui un artefatto software tipicamente è soggetto nel corso del suo naturale ciclo di vita
- Il costo delle modifiche che vengono evidenziate da questo tipo di test è normalmente basso
 - I benefici che la scrittura di test ben fatti portano alla qualità del codice sono alti e compensano abbondantemente nel medio termine i maggiori costi dovuti alla scrittura dei test

Test di integrazione

- Un test di integrazione valuta il comportamento all'interfaccia di due moduli software
 - Si focalizza sul determinare la correttezza di tale interfaccia, evidenziando le incongruenze tra le parti che stanno interagendo
- Come nel caso dei test di unità, è costituito da frammenti di codice che sollecitano l'interazione tra due parti
 - A differenza dei test di unità, tuttavia, ignora volutamente la struttura interna dei moduli che stanno interagendo, assumendo il loro corretto comportamento - sul piano individuale - alle specifiche definite
 - Per questo motivo viene eseguito solo su oggetti che hanno correttamente superato il test di unità
- I costi delle modifiche che occorre introdurre se un test di integrazione fallisce sono generalmente più elevati
 - In quanto richiedono il ridisegno di una o entrambe le parti che interagiscono
- I test di integrazione sono co-progettati da sviluppatori e tester
 - Vengono eseguiti dai tester

Test complessivi (*end-to-end*)

- Il **test di sistema** valuta il comportamento del prodotto finito
 - Sia nei suoi aspetti funzionali (cosa deve/non deve fare) che non funzionali (sicurezza, affidabilità, resilienza, prestazioni, scalabilità, ...)
- Basato su modelli di comportamento tratti da casi d'uso reali
 - Progettato ed eseguito da tester, allo scopo di non farsi influenzare dalle assunzioni fatte durante lo sviluppo
- Il test di accettazione verifica che il sistema sia conforme alle aspettative del committente/cliente
 - Eseguito dagli utenti finali

Test di unità (Rust)

- I **test di unità** possono essere scritti nel modulo da testare o, meglio, in un suo sotto-modulo chiamato convenzionalmente 'tests'
 - Questo facilita l'ispezione del codice quando il numero di test cresce significativamente
- Il (sotto-)modulo contenente i test è preceduto dall'annotazione **`#[cfg(test)]`**
 - Questo informa il compilatore che il codice contenuto al suo interno deve essere incluso solo quando la compilazione avviene con il comando **`cargo test`**

Sintassi dei test

```
// funzione da collaudare

pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
    }
}
```



Sintassi dei test

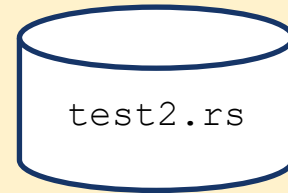
- La struttura generale di una funzione etichettata con **#[test]** è la seguente:
 - **Arrange** - Preparazione dei valori su cui operare
 - **Act** - Esecuzione del codice da testare
 - **Assert** - Verifica, mediante asserzioni, che i risultati ottenuti siano quelli attesi
- Per supportare l'ultimo passo, la libreria standard di Rust mette a disposizione varie macro procedurali
 - **assert!(*boolean_condition*)** - impone che la condizione sia vera
 - **assert_eq!(*value1*, *value2*)** - impone l'eguaglianza tra i due valori
 - **assert_ne!(*value1*, *value2*)** - impone la disuguaglianza tra i due valori
- Per verificare che un'espressione generi la condizione di panico, è possibile annotare il test anche con le annotazioni:
 - **#[should_panic]**, viene verificato che la funzioni generi un panic!
 - **#[should_panic(expected = "msg")]**, viene verificato che il messaggio generato da panic! contenga "msg"


```
pub fn divide(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        Err(String::from("Divisione per zero"))
    } else {
        Ok(a / b)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_divide_ok() {
        assert_eq!(divide(10, 2), Ok(5));
    }

    #[test]
    #[should_panic(expected = "Divisione per zero")]
    fn test_divide_panic() {
        divide(10, 0).unwrap();
    }
}
```

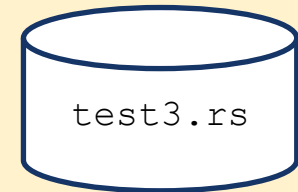


Assert con Result<T, Error>

- Se una funzione di test restituisce un valore di tipo **Result<T, Error>**, è possibile utilizzare la condizione di errore per indicare che il test è fallito

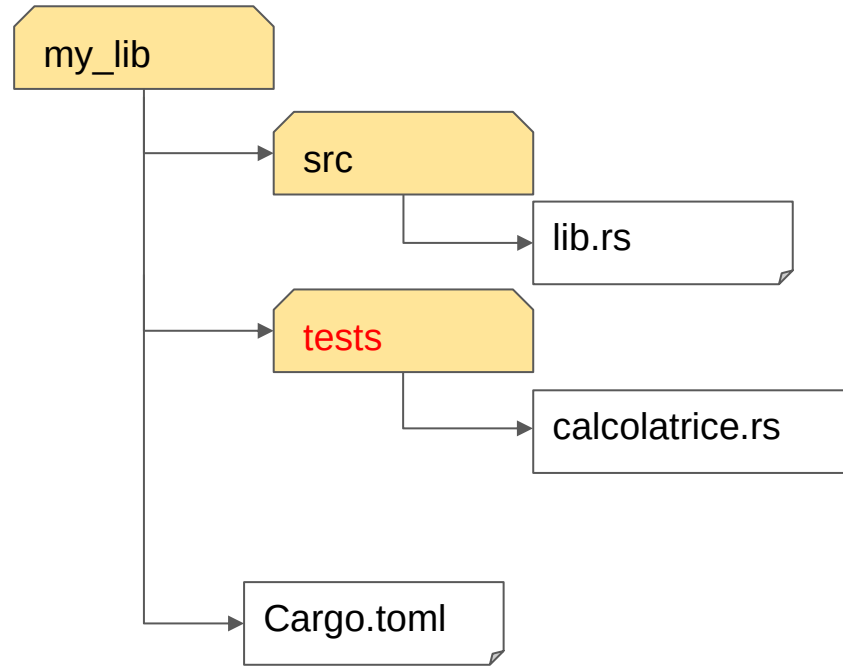
```
// Funzione che potrebbe fallire
fn parse_number(s: &str) -> Result<i32, String> {
    s.parse().map_err(|_| "Parsing fallito".to_string())
}

#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn test_parse_valid() -> Result<(), String> {
        let num = parse_number("42")?; // Usa ? per propagare l'errore
        assert_eq!(num, 42);
        Ok(()) // Test passa se tutto è Ok
    }
    #[test]
    fn test_parse_invalid() -> Result<(), String> {
        let result = parse_number("abc");
        assert!(result.is_err());
        Ok(())
    }
}
```



Test di integrazione (Rust)

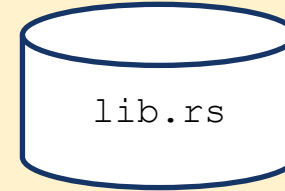
- Servono per testare il comportamento del crate dall'esterno come se fosse usato da qualcun altro
- Vivono in un posto separato: Sono contenuti in una cartella separata denominata **tests**, a lato della cartella **src** del progetto
- I file sorgente all'interno della cartella di test importano i simboli pubblici oggetto di valutazione tramite istruzioni di tipo **use...**
- Solo le **librerie** possono avere test di integrazione



```
#[derive(Debug, PartialEq)]
pub enum CalcoloErrore {
    DivisionePerZero,
}

pub fn somma(a: i32, b: i32) -> i32 {
    a + b
}

pub fn dividi(a: i32, b: i32) -> Result<i32, CalcoloErrore> {
    if b == 0 {
        Err(CalcoloErrore::DivisionePerZero)
    } else {
        Ok(a / b)
    }
}
```



```
use my_lib::*;

#[test]
fn test_somma() {
    assert_eq!(somma(10, 5), 14);
    assert_eq!(somma(-2, 3), 1);
}

#[test]
fn test_divisione_ok() {
    assert_eq!(dividi(10, 2), Ok(5));
}

#[test]
fn test_divisione_per_zero() {
    assert_eq!(dividi(10, 0), Err(CalcoloErrore::DivisionePerZero));
}
```



Eseguire i test

- Il comando **cargo test** ricompila il programma con la direttiva “test” ed esegue tutti i test contenuti nel package
 - Compresi quelli eventualmente presenti all’interno della documentazione
 - Viene prodotto un report dettagliato che indica ogni singolo fallimento, dove viene evidenziato il risultato atteso e quello realmente ottenuto seguito da un riassunto con il nome di tutte le funzioni di test che sono fallite
- E’ possibile lanciare solo alcuni test, specificando il nome della funzione (o una sua sottoparte) come ulteriore parametro del comando
 - **cargo test add_** esegue tutti i test i cui nomi contengono “add_”
- Se uno o più test è decorato con l’attributo **#[ignore]**, viene normalmente tralasciato dall’esecuzione
 - È possibile includere tali test con il comando **cargo test -- --ignored**

Automatizzare la creazione di test

- Normalmente, le funzioni di test sono prive di parametri, dovendo poter essere invocate direttamente dal motore di test
 - La libreria **rstest** (<https://github.com/la10736/rstest>) permette di creare test parametrici, costituiti cioè da funzioni dotate parametri in ingresso
 - Tali funzioni devono essere precedute dall'attributo **#[rstest]**
- I parametri possono essere forniti in due modi alternativi:
 - Tramite un elenco di tuple alternative, descritte tramite l'attributo **#[case(...)]**
 - Tramite funzioni di set-up, annotate con l'attributo **#[fixture]**, il cui nome deve coincidere con il nome del parametro di ingresso presente nei test, il cui risultato viene usato come parametro.


```
use rstest::rstest;
use my_lib::*;
```

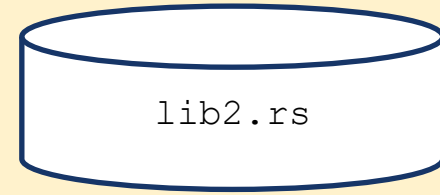
```
#[rstest]
#[case(2, 3, 5)]
#[case(-1, 1, 0)]
#[case(0, 0, 0)]
fn test_somma(#[case] a: i32, #[case] b: i32, #[case] expected: i32) {
    assert_eq!(somma(a, b), expected);
}

#[rstest]
#[case(10, 2, Ok(5))]
#[case(1, 0, Err(CalcoloErrore::DivisionePerZero))]
fn test_divisione(#[case] a: i32, #[case] b: i32, #[case] expected:
Result<i32, CalcoloErrore>) {
    assert_eq!(dividi(a, b), expected);
}
```



calcolatrice2.rs

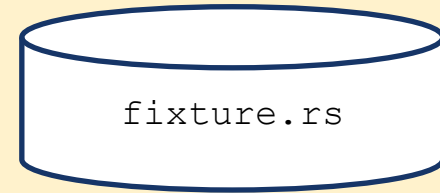
```
//src/lib2.rs
pub fn somma_vec(v: &[i32]) -> i32 {
    v.iter().sum()
}
```



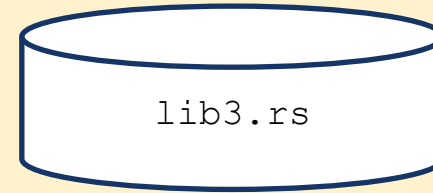
```
//tests/fixture.rs
use rstest::{fixture, rstest};
use my_lib2::somma_vec;

#[fixture]
fn vettore() -> Vec<i32> {
    vec![1, 2, 3, 4]
}

#[rstest]
fn test_somma_vec(vettore: Vec<i32>) {
    assert_eq!(somma_vec(&vettore), 10);
}
```



```
//src/lib3.rs
pub fn somma_vec(v: &[i32]) -> i32 {
  v.iter().sum()
}
```



```
use rstest::{fixture, rstest};
use my_lib3::scala_vec;
```

```
#[fixture]
fn base_vec() -> Vec<i32> {
  vec![1, 2, 3]
}
```

```
#[rstest]
#[case(2, vec![2, 4, 6])]
#[case(0, vec![0, 0, 0])]
#[case(-1, vec![-1, -2, -3])]
fn test_scala_vec(
  base_vec: Vec<i32>,           // fixture
  #[case] fattore: i32,        // parametro
  #[case] atteso: Vec<i32>,    // parametro atteso
) { assert_eq!(scala_vec(&base_vec, fattore), atteso);}
```





Per saperne di più

- Testing in Rust
 - <https://anismousse.medium.com/testing-in-rust-22f27136b433>
- Writing Automated Tests
 - <https://doc.rust-lang.org/book/ch11-00-testing.html>
- Rust Mock Shootout!
 - https://asomers.github.io/mock_shootout/
- Formatting, Linting, and Documenting with Rust
 - <https://blog.devgenius.io/formatting-linting-and-documenting-with-rust-eb7b189ade65>